

# NEURAX

The analytical compiler

What a neural network will cost,  
derived from its architecture — without running it.

---

Eleven dialects, from a JSON description to a costed report.  
This talk is about the compiler, and only the compiler.

# What this talk covers

And, deliberately, what it does not

---

## In scope

---

- ▶ The intermediate representation and its eleven passes
- ▶ How parameters, FLOPs, memory, latency and cost are derived
- ▶ The analytical formula library
- ▶ How the results are validated against published models
- ▶ Where the model is approximate, and by how much

## Out of scope

---

- ▶ The visual studio and its workspaces
- ▶ Hardware detection and measurement
- ▶ The training runtime
- ▶ Code generation
- ▶ The assistant

Each is a talk of its own. The compiler is the part every one of them is built on top of.

# The question

Asked before the money is spent, not after

“We are considering a 70B de-  
coder with grouped-query attention.  
Will it fit? How long will it train? What will it cost?”

**TODAY**

## Run a smaller version

Hours of GPU time to answer a question about a model you did not build. Extrapolating from it is a second guess on top of the first.

NEURAX

**TODAY**

## Ask someone who has

Fast, unreproducible, and specific to their hardware, their batch size and their framework version.

**NEURAX**

## Derive it

The cost of a network is a function of its shape. Shapes are known before any weight exists.

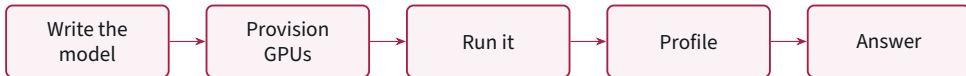
The problem

# Measure or derive

Two answers to the same question, with very different economics

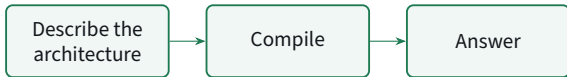
## MEASURE

hours to days



## DERIVE

sub-millisecond



The measured answer is the ground truth — and it is only available *after* you have already paid for it. NEURAX gives up exactness to answer the question while it is still a question.

# What an analytical compiler is

The contract, stated plainly

---

**It takes** a description of an architecture: layer types, shapes, hyperparameters, a target device.

**It returns** parameters, FLOPs, peak memory, latency, throughput, scaling behaviour, energy and money.

**It never executes the model.** No tensor is allocated, no kernel is launched, no weight exists.

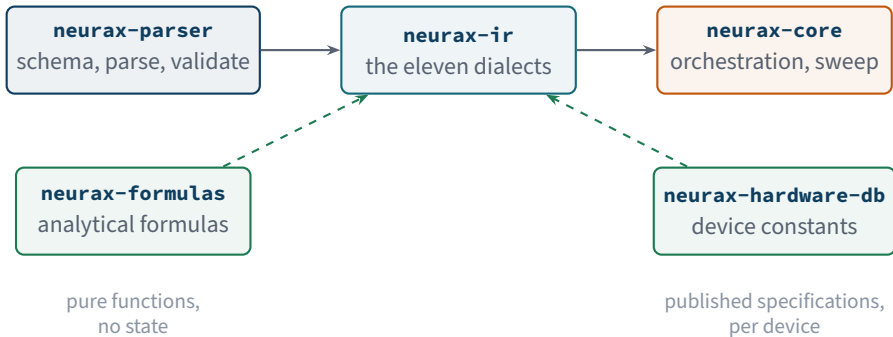
Everything it reports is derived from the shape of the computation and a table of hardware constants.

## Consequences of never running

- + answers exist before hardware does
- + cost is independent of model size
- + thousands of variants can be swept
- + fully deterministic and reproducible
- no convergence, no accuracy
- no kernel-level scheduling effects
- bounded by the fidelity of its formulas

# How the compiler is built

Five crates, one direction of dependency



The two dashed crates are leaves: they know nothing about the pipeline, which is what makes each of them testable against published figures on its own. **neurax-formulas** is a library of pure functions — given the same shape, it returns the same count, forever.

# The front end

Before any analysis, a description that is known to be well-formed

```
{
  "schema_version": "1.0",
  "model": {
    "name": "tiny-gpt",
    "type": "transformer",
    "layers": [ {
      "id": "attn_0",
      "layer_type": "attention",
      "input_shape": [128, 768],
      "params": { "num_heads": 12 }
    } ],
    "global_params": { "hidden_size": 768
  },
  "training": { "batch_size": 32, ... },
  "hardware": { "gpus": [ ... ] }
}
```

## Three inputs, not one

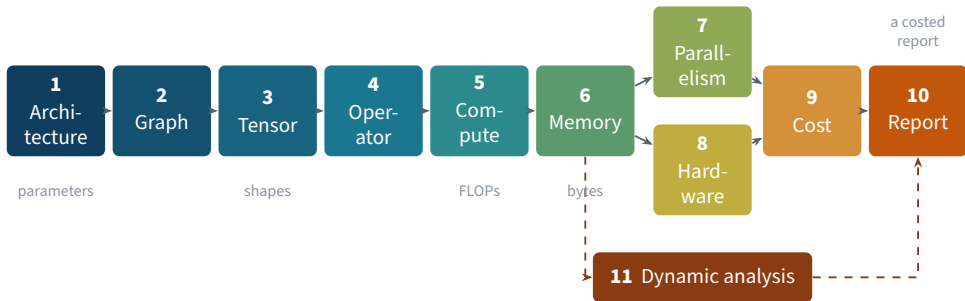
A model is not costable on its own. The same architecture on a different device, at a different batch size, is a different answer — so the device and the training regime are part of the input, not an afterthought.

## Validation rejects before it analyses

Zero attention heads, a hidden size that is not divisible by the head count, shapes that cannot connect. A compiler that produced a confident number for an impossible model would be worse than one that refused.

# The pipeline

Eleven dialects, each lowering the description one step closer to money



Each pass owns one question and may only read what the passes before it produced. Nothing reaches back — which is what makes an eleven-stage derivation auditable: every number has exactly one place it could have come from.



# The contract every pass signs

Three methods, and the third is the interesting one

```
pub trait IrPass {  
  type Input;  
  type Output;  
  type Metrics;  
  
  fn build(&self, input, ctx)  
    → Result<Output>;  
  
  fn compute_metrics(&self, out, ctx)  
    → Result<Metrics>;  
  
  fn validate(&self, out, metrics)  
    → Result<()>;  
}
```

**build** lowers the previous dialect into this one.

**compute\_metrics** derives this pass's numbers from the lowered form.

**validate** checks them *before* they are allowed downstream — peak memory greater than zero, forward FLOPs greater than zero, shapes that connect.

A wrong number caught in the pass that produced it names that pass. The same number caught three passes later names nothing, and looks like a cost bug.

# Phases 1–2: Architecture and Graph

From a list of layers to a directed acyclic graph

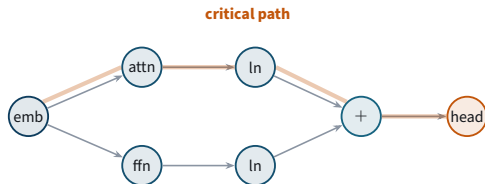
## PHASE 1 Architecture

Normalises the description into typed layer definitions and resolves repetition: a decoder stated as “32 identical blocks” becomes 32 blocks with real widths.

**Produces:** total parameters, layer count.

## PHASE 2 Graph

Builds the computational DAG, checks it is acyclic, and finds the critical path — the longest chain of dependent operations, which is what bounds latency no matter how much hardware you add.

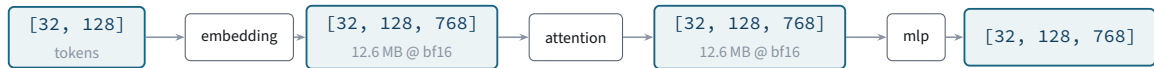


Fifty-five layer types are recognised, across transformers, CNNs, state-space models, mixtures of experts, RNNs, GNNs, GANs and diffusion.

# Phase 3: Tensor

Shape inference — every intermediate tensor, before any of them exists

---



## Why shapes come before everything

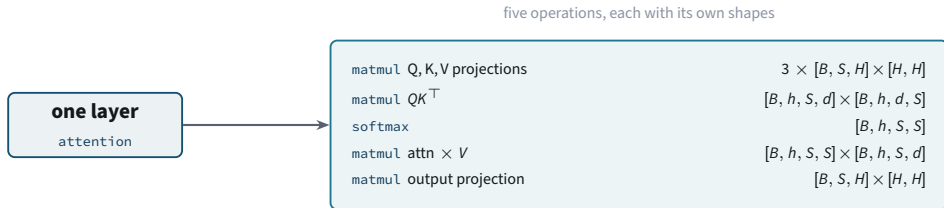
A FLOP count is a function of the shapes a kernel operates on. A byte count is a function of the shapes that are live. Neither question can be asked until every intermediate shape is known.

## Where it refuses

If two shapes cannot connect, this is the pass that says so, naming the two layers. The alternative — silently reshaping — would produce a complete, confident and entirely fictional report.

## Phase 4: Operator

From layers to the operations a device actually issues



This is the pass that makes the rest arithmetic. “An attention layer” has no FLOP count; five matmuls of known shape do. Fusion opportunities are identified here too — adjacent elementwise operations a real kernel would not round-trip through memory.

## Phase 5: Compute

FLOPs, derived — shown for attention, because it is the one that surprises

$$QKV = 3 \cdot 2BSHw$$

$$QK^T = 2BhS^2d$$

$$\text{attn} \cdot V = 2BhS^2d$$

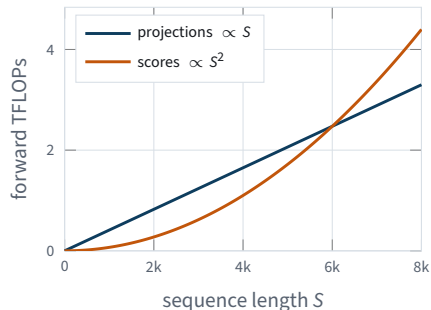
$$\text{softmax} \approx 5BhS^2$$

$$\text{out} = 2BSwH$$

$B$  batch,  $S$  sequence,  $H$  hidden,  $h$  heads,  $d$  head dimension,  
 $w = h \cdot d$ .

Causal attention halves the three  $S^2$  terms — and only those three, because the projections are not masked.

Backward is taken as  $2 \times$  forward; the optimizer step adds  $\approx 10\%$ .



One 4096-wide layer. The crossing point is where context length stops being free — worth answering before training, not after.

# The formula library

Fifty-five layer types, each an independently checkable pure function

---

## attention

MHA, GQA, MQA,  
sliding-window,  
cross-attention

## mlp / moe

dense, gated,  
top-*k* routing,  
shared experts

## conv

2D, bottleneck,  
MBConv, ConvNeXt,  
dense blocks

## ssm

Mamba, S4, H3,  
RWKV, retention

## rnn / gnn

LSTM, GRU, bidir,  
GCN, GAT, R-GCN

## diffusion

U-Net, VAE,  
time embedding

## Pure, by construction

No state, no context, no I/O.

`attention_flops(B, S, H, h, causal)` returns the same number today and in a year. That is what makes each one checkable against a published count in isolation.

## Precision is a parameter, not a footnote

`dtype_bytes` returns `f64`, not an integer, because `int4` packs two values per byte. Returning 1 reported `int4` as costing the same as `int8` — twice its real footprint, for exactly the quantisation people reach for to save memory.

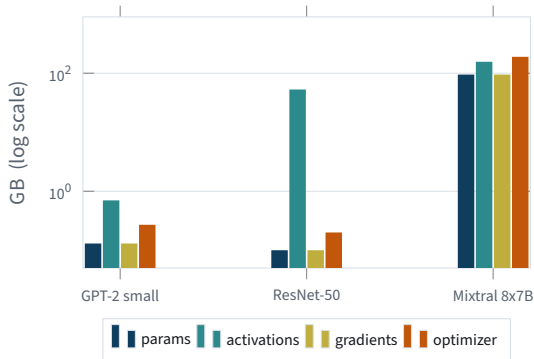
# Phase 6: Memory

Four terms, and the one everybody forgets

$$M_{\text{peak}} = \underbrace{Pb}_{\text{params}} + \underbrace{A}_{\text{activations}} + \underbrace{Pb}_{\text{gradients}} + \underbrace{kP}_{\text{optimizer}}$$

$P$  parameters,  $b$  bytes per element,  $k$  the optimizer's state multiplier — Adam keeps two moments in fp32, so  $k = 8$  bytes per parameter whatever the training precision.

A 7B model in bf16 is 14 GB of weights and 70 GB of training state. Sizing a run from the weight count alone is how a job dies at step one on a card that looked ample.



Computed by NEURAX. Peak: 1.24, 53.4, 535 GB.

The passes

# Phase 7: Parallelism

What happens when one device is not enough

## Sharding

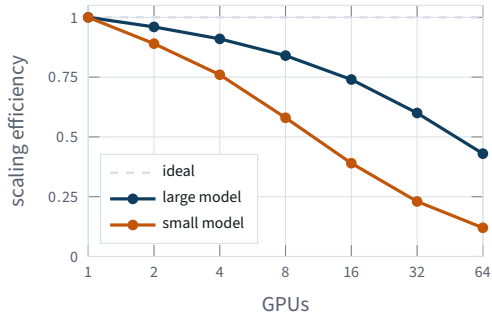
ZeRO stages 0–3 progressively shard the optimizer state, then the gradients, then the parameters themselves. Each stage reduces memory per device and changes what has to be communicated.

## Communication

A data-parallel step must all-reduce one buffer the size of the gradients — not the model's whole memory traffic. ZeRO-3 in particular *increases* communication: it must gather sharded parameters back on demand.

Scaling efficiency is derived, not assumed.

The pass reports the GPU count past which



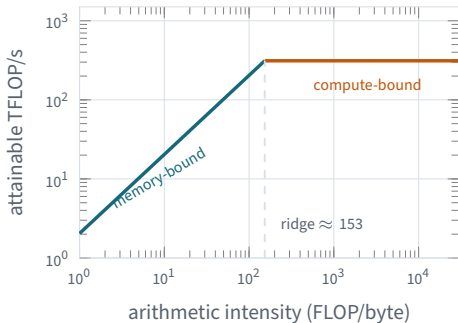
Shape of the curve the pass produces: communication is a fixed cost per step, so the smaller the compute per device, the sooner it dominates. Illustrative shape; the pass

computes the real curve per model.  
The passes



# Phase 8: Hardware

The roofline — and why latency is not simply the larger of two numbers



A100-80GB: 312 TFLOP/s fp16, 2039 GB/s. NEURAX computes the ridge as peak  $\times$  efficiency / bandwidth.

NEURAX

## Latency, as modelled

$$t = \max(t_c, t_m) + \min(t_c, t_m)(1 - \rho) + t_{\text{comm}} + n_k \tau$$

$t_c$  compute,  $t_m$  memory,  $\rho$  overlap,  $n_k$  kernels,  $\tau$  per-launch overhead.

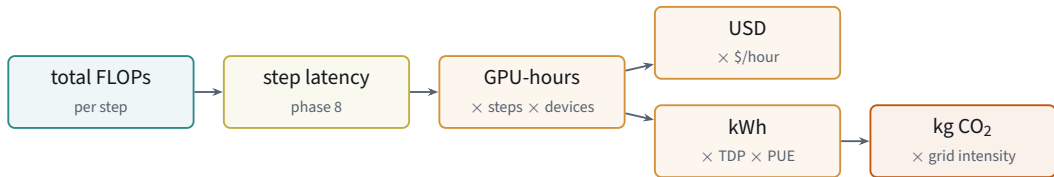
Taking  $\max(t_c, t_m)$  alone assumes compute and memory overlap perfectly. No real GPU does. NEURAX uses  $\rho = 0.3$  and adds  $5 \mu\text{s}$  per kernel launch — for a model issuing thousands of small kernels, not a rounding error.

Bottlenecks carry a dead band too: compute- or memory-bound only past  $1.5\times$ , otherwise *balanced*.

The passes

# Phases 9–10: Cost and Report

Where FLOPs become hours, money, kilowatt-hours and carbon



## The report is not a formatting step

It assembles every pass's metrics into one document and runs the diagnostic rules across the whole picture — checks that need more than one pass's output to be possible.

## Diagnostics carry a code and a suggestion

E001–E005 errors, W001–W009 warnings, H001–H008 hints, I001–I003 information. Each has a severity, an optional layer to point at, and an estimate of its impact on the report's own precision.

# Phase 11: Dynamic analysis

Three questions that are about behaviour rather than arithmetic

---

## VIRTUAL MEMORY

Allocation behaviour over a step: liveness intervals, the fragmentation a real allocator would suffer, and the largest batch that still fits.

## STABILITY

Numerical risk read off the graph and the precision: depth without normalisation, activation ranges that overflow fp16, initialisation that will not hold across the reported depth.

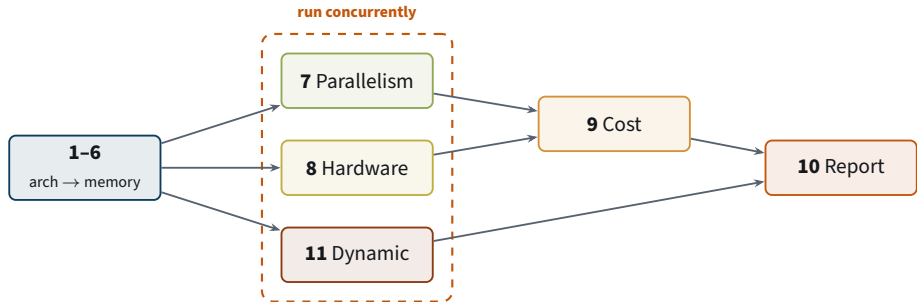
## BEHAVIOURAL

Synthesises the step profile from the compute IR — which operations dominate, and what the shape of a step actually looks like.

Its stability output feeds a hyperparameter hint in the report ([H007](#)), which is why it must finish before the report is closed — but it reads neither cost nor hardware, so it does not have to wait for them.

# The compiler's own dependency graph

Eleven passes, but not eleven serial steps



Phases 7, 8 and 11 read only what phases 1-6 produced, and nothing from each other — so they run in parallel. Dynamic analysis used to wait for Cost and Report even though it never touches their output: dead serialisation, removed by reading the dependency graph rather than the phase numbering.

# Does it agree with reality?

Eleven reference models against their published parameter counts

| Model            | Published | NEURAX  | Error  |
|------------------|-----------|---------|--------|
| GCN (ogbn-arxiv) | 110 120   | 110 120 | +0.00% |
| DCGAN            | 6.342 M   | 6.340 M | −0.03% |
| ResNet-50        | 25.6 M    | 25.59 M | −0.04% |
| GPT-3 175B       | 175.0 B   | 174.6 B | −0.24% |
| VGG-16           | 138 M     | 138.4 M | +0.26% |
| AWD-LSTM (PTB)   | 24 M      | 24.20 M | +0.84% |
| Mixtral 8×7B     | 46.7 B    | 47.39 B | +1.47% |
| LLaMA-2 70B      | 70.0 B    | 68.72 B | −1.82% |
| RWKV-4 7B        | 7.5 B     | 7.19 B  | −4.17% |
| Mamba 2.8B       | 2.8 B     | 2.66 B  | −4.89% |
| DeepSeek-V3      | 671 B     | 701.4 B | +4.53% |

Sources cited per row in the test itself: papers where a count is published, independent re-derivation where it is not. Published counts are themselves approximate — they disagree on tied embeddings and on whether biases are counted

# The test that found four wrong answers

Written to check the thing it claimed to check

| Model        | Published | First measured | Error  |
|--------------|-----------|----------------|--------|
| LLaMA-2 70B  | 70.0 B    | 49.9 B         | −28.7% |
| Mixtral 8×7B | 46.7 B    | 103.8 B        | +122%  |
| DeepSeek-V3  | 671 B     | 1 395 B        | +108%  |
| RWKV 7B      | 7.5 B     | 0.24 B         | −96.7% |

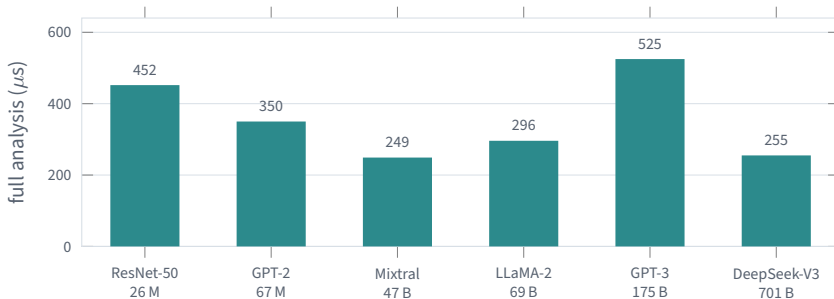
## Three causes, all structural

- A scale derived from the attention-block count was applied to every repeating kind, so a config listing 2 attention and 4 mixture blocks got twice the mixture layers it asked for.
- `shared_experts` defaulted to 1, adding an expert to every mixture that never mentioned one.
- RWKV was costed with Mamba’s formula.

A test named `modelTemplates.accuracy.test.ts` claimed these models “must reproduce their published parameter counts”. It checked parameter *name aliasing*. Nothing anywhere compared a computed figure to a known one — so all four errors were invisible while the

# What it costs to ask

One machine, release build, best of 25 runs each



A 701-billion-parameter model is analysed in **255 microseconds** — faster than a 26-million-parameter one. Cost tracks the number of *layer definitions*, not parameters, because nothing is ever allocated.

# What the compiler does not know

Stated here so it is not discovered later

## Outside the model entirely

**Convergence.** It reports what a step costs, never whether the model learns. No accuracy, no loss curve.

**Kernel scheduling.** Real occupancy, cache behaviour and launch ordering are represented by two constants, not simulated.

**Framework overhead.** Python, the data loader, and checkpoint I/O are not in the model.

## Approximations it makes on purpose

**Backward** =  $2 \times$  **forward**. Standard, and wrong in the third digit for architectures with heavy recomputation.

**Optimizer**  $\approx 10\%$ . A constant, not a derivation.

**Efficiency factors.** Per-operation, per-device multipliers from a calibration table — the one place measured data enters the model.



# In summary

---

## The compiler

- ▶ Eleven dialects, from a JSON architecture to a costed report
- ▶ Every pass builds, measures and *validates* before the next one may read it
- ▶ Fifty-five layer types, as pure functions checkable in isolation
- ▶ Nothing is ever executed — no tensor, no kernel, no weight

## What that buys

- ▶ Parameter counts within a few percent of eleven published models
- ▶ A full analysis in a few hundred microseconds, at any model size
- ▶ Answers before the hardware is provisioned, not after
- ▶ A cost that can be swept across thousands of configurations